



Universitatea
Transilvania
din Brașov
FACULTATEA DE MATEMATICĂ
ȘI INFORMATICĂ

LUCRARE DE LICENȚĂ

MediLink: Platformă Medicală Digitală

Conducător științific:

Lect. Dr. Bocu Răzvan

Absolvent:

Rădoi Vlad Cristian

Brașov, 2026

Cuprins

1	Introducere	2
1.1	Contextul și motivația lucrării	2
1.2	Studiu comparativ cu aplicațiile existente	2
1.2.1	Aplicații internaționale	3
1.2.2	Aplicații românești	3
1.2.3	Analiza comparativă	3
1.3	Prezentarea aplicației MediLink	4
1.3.1	Funcționalități pentru pacienți	4
1.3.2	Funcționalități pentru medici	5
1.3.3	Funcționalități pentru asistenți medicali	5
1.3.4	Funcționalități administrative	6
1.4	Structura lucrării	6
1.5	Contribuții originale	6
2	Noțiuni teoretice și tehnologii utilizate	8
2.1	Arhitectura aplicațiilor web moderne	8
2.1.1	Modelul client-server și arhitectura REST	8
2.1.2	Arhitectura Single Page Application (SPA)	8
2.2	FastAPI — Framework-ul backend	9
2.2.1	Prezentare generală	9
2.2.2	Caracteristici distinctive	9
2.2.3	Dependency Injection în MediLink	10
2.3	Angular 19 — Framework-ul frontend	10
2.3.1	Prezentare generală	10
2.3.2	Componente Standalone	11
2.3.3	Angular Material Design 3	11
2.4	PostgreSQL — Baza de date	12
2.4.1	Prezentare generală	12
2.4.2	SQLAlchemy ORM	12
2.4.3	Alembic — Migrații ale schemei	13
2.5	Redis — Cache și sesiuni	13
2.6	Securitate	13

2.6.1	Autentificare JWT	13
2.6.2	Autentificare în doi factori (2FA/TOTP)	14
2.6.3	Criptarea datelor medicale sensibile	14
2.6.4	RBAC — Control al accesului bazat pe roluri	14
2.7	Inteligența artificială în medicină	14
2.7.1	Large Language Models (LLM)	14
2.7.2	Aplicații AI în MediLink	15
2.8	Comunicare în timp real	15
2.8.1	WebSockets	15
2.8.2	WebRTC — Teleconsultații video	15
2.9	Containerizare cu Docker	16
2.10	GDPR și protecția datelor medicale	16
3	Contribuții personale	18
3.1	Arhitectura generală a sistemului	18
3.2	Modelul de date	18
3.2.1	Entitățile principale	18
3.2.2	Cheile primare UUID	19
3.2.3	Criptarea transparentă — EncryptedString	19
3.3	API REST — Structura și implementarea	20
3.3.1	Organizarea endpoint-urilor	20
3.3.2	Rate Limiting	21
3.4	Sistemul de autentificare și securitate	21
3.4.1	Fluxul de autentificare	21
3.4.2	Middleware-ul de autentificare	22
3.4.3	Middleware RBAC	22
3.5	Modulul de fișe medicale	23
3.5.1	Tipologia înregistrărilor medicale	23
3.5.2	Detectarea automată a anomaliilor	23
3.5.3	Paginarea și filtrarea	24
3.6	Modulul de semne vitale	24
3.6.1	Parametrii monitorizați	24
3.6.2	Vizualizarea grafică	25
3.6.3	Alertele automate	25
3.7	Sistemul de programări	25
3.7.1	Ciclul de viață al unei programări	25
3.7.2	Remindere automate	25
3.8	Inteligența artificială — Implementare detaliată	26
3.8.1	Chat medical AI	26
3.8.2	Predicția riscului medical	26

3.9	Teleconsultații video — WebRTC	27
3.9.1	Arhitectura semnalizării	27
3.10	Mesageria în timp real	28
3.10.1	Arhitectura WebSocket	28
3.10.2	Serviciul Angular de WebSocket	28
3.11	Exportul GDPR	28
3.12	Interfața cu utilizatorul	29
3.12.1	Design system — Angular Material M3	29
3.12.2	Diagrama fluxului de navigare	29
3.12.3	Responsive design	29
3.13	Testarea aplicației	29
3.13.1	Teste unitare backend	29
3.13.2	Rularea testelor	30
4	Concluzii și perspective de dezvoltare	31
4.1	Concluzii	31
4.2	Limitări actuale	32
4.3	Perspective de dezvoltare	32
4.3.1	Integrarea dispozitivelor medicale IoT	32
4.3.2	Diagnosticare asistată AI prin imagini medicale	32
4.3.3	Analiză predictivă populațională	33
4.3.4	Aplicație mobilă	33
4.3.5	Conformitate HL7 FHIR și Spațiul European al Datelor de Sănătate	33
4.3.6	Sistem de telemedicină asincronă	33
	Bibliografie	34

Listă de figuri

3.1	Arhitectura generală a platformei MediLink	18
-----	--	----

Listă de tabele

1.1	Comparație MediLink cu aplicații similare	4
3.1	Entitățile bazei de date MediLink	19
3.2	Grupurile de endpoint-uri ale API-ului MediLink	21

1. Introducere

1.1 Contextul și motivația lucrării

Transformarea digitală a sistemului de sănătate reprezintă una dintre cele mai importante tendințe globale ale deceniului curent. Potrivit raportului Organizației Mondiale a Sănătății din 2023 privind sănătatea digitală globală [1], digitalizarea serviciilor medicale reduce erorile clinice cu până la 30%, scurtează timpii de așteptare și îmbunătățește semnificativ comunicarea medic-pacient. La nivel european, Strategia de Sănătate Digitală a Uniunii Europene 2021-2025 [2] stabilește ca obiectiv prioritar crearea unui Spațiu European al Datelor de Sănătate, care să permită accesul securizat și interoperabil la dosarele medicale electronice.

În România, gradul de digitalizare al sistemului sanitar rămâne unul dintre cele mai scăzute din Uniunea Europeană. Conform datelor Ministerului Sănătății [3], mai puțin de 15% dintre medicii de familie utilizează sisteme electronice integrate pentru gestionarea pacienților, majoritatea activităților desfășurându-se pe suport hârtie sau cu instrumente disperate, neintegrate. Această realitate generează numeroase probleme: pierderea istoricului medical al pacienților, imposibilitatea accesului de urgență la informații critice (alergii, medicație curentă), lipsa comunicării eficiente între specialiști și, nu în ultimul rând, o experiență precară a pacientului în relația cu sistemul de sănătate.

MediLink se naște din nevoia concretă de a adresa aceste probleme printr-o soluție modernă, completă și accesibilă, care să poată fi adoptată rapid de unitățile medicale de dimensiuni mici și medii — cabinete medicale, policlinici și centre medicale private — fără a necesita investiții infrastructurale majore, grație arhitecturii bazate pe containere Docker.

Alegerea acestei teme este motivată și de evoluția rapidă a inteligenței artificiale aplicate în medicină. Modelele lingvistice mari (*Large Language Models* — LLM) au demonstrat în studii recente capacitatea de a asista clinicienii în formularea diagnosticilor diferențiale [4], de a interpreta rezultate de laborator și de a genera recomandări terapeutice bazate pe dovezi. Integrarea acestor capabilități într-o platformă medicală accesibilă reprezintă un pas concret spre medicina personalizată și asistată de AI.

1.2 Studiu comparativ cu aplicațiile existente

Piața soluțiilor software pentru gestionarea activității medicale este diversificată, însă majoritatea produselor existente prezintă limitări semnificative din perspectiva integrării, a

accesibilității financiare sau a suportului pentru funcționalitățile moderne discutate anterior.

1.2.1 Aplicații internaționale

Epic Systems [5] este considerată cea mai completă platformă de tip Electronic Health Record (EHR) la nivel mondial, utilizată de peste 250 de milioane de pacienți în Statele Unite. Oferă funcționalități avansate de gestionare a dosarelor medicale, integrare cu dispozitive medicale IoT și module de analiză predictivă. Principalele dezavantaje sunt costul prohibitiv (implementarea unei instanțe complete depășind 100 de milioane de dolari pentru un spital mare), complexitatea ridicată a implementării și lipsa suportului nativ pentru legislația europeană GDPR.

Doctolib [6], popular în Franța, Belgia și Germania, oferă un sistem de programări online și teleconsultații, dar se concentrează exclusiv pe gestionarea agendei medicale, lipsind funcționalitățile de dosar medical electronic, semne vitale sau AI integrat.

Practo [7], utilizat preponderent în India și Asia de Sud-Est, oferă un ecosistem similar cu MediLink, incluzând dosare medicale și teleconsultații. Limitările principale constau în absența conformității cu GDPR, infrastructura cloud centralizată (fără opțiune de deployment local/privat) și lipsa integrării AI generative.

1.2.2 Aplicații românești

Hipocrate [8] este cel mai utilizat sistem de gestionare a activității medicale din România, cu focus pe cabinete medicale de familie și specialitate. Deși acoperă bine nevoile de bază (programări, rețete, scrisori medicale), aplicația are o interfață învechită, nu oferă funcționalități de teleconsultații, mesagerie integrată sau AI, și nu pune la dispoziție un portal dedicat pacientului.

MedicalSoft [9] și **MedEVI** sunt soluții similare Hipocrate, orientate în principal spre automatizarea raportărilor către Casa Națională de Asigurări de Sănătate (CNAS), fără accent pe experiența pacientului sau pe funcționalități digitale moderne.

1.2.3 Analiza comparativă

Tabelul 1.1 prezintă o analiză comparativă a principalelor funcționalități ale aplicațiilor menționate față de MediLink:

Tabela 1.1: Comparație MediLink cu aplicații similare

Funcționalitate	MediLink	Epic	Doctolib	Practo	Hipocrate
Dosar medical electronic	?	?	?	?	?
Programări online	?	?	?	?	?
Teleconsultații video	?	?	?	?	?
Mesagerie în timp real	?	?	?	?	?
Semne vitale + grafice	?	?	?	?	?
Chat AI medical	?	Parțial	?	?	?
Predicție risc AI	?	?	?	?	?
Export GDPR	?	?	?	?	?
2FA/TOTP	?	?	?	?	?
Criptare date sensibile	?	?	?	Parțial	?
Portal pacient dedicat	?	?	?	?	?
Open source / Docker	?	?	?	?	?
Cost accesibil	?	?	?	?	?

Analiza comparativă evidențiază că MediLink se distinge prin combinația unică de funcționalități avansate (AI integrat, WebRTC, WebSockets), conformitate completă cu GDPR, arhitectură containerizată Docker (care permite deployment privat, fără dependență de cloud-ul unui furnizor terț) și accesibilitate — aspecte care nu se regăsesc simultan în niciuna dintre soluțiile existente analizate.

1.3 Prezentarea aplicației MediLink

MediLink este o aplicație web full-stack care implementează un ecosistem medical digital complet pentru patru categorii de utilizatori: **pacienți, medici, asistenți medicali și administratori**.

1.3.1 Funcționalități pentru pacienți

Pacienții beneficiază de un portal personal complet, care le oferă acces la:

- **Dashboard personalizat** cu statistici de sănătate, programările viitoare și ultimele notificări;
- **Fișă medicală electronică** structurată pe tipuri de înregistrări: consultații, analize de laborator, tratamente, investigații paraclinice și rețete;

- **Semne vitale** cu posibilitatea de vizualizare a evoluției în timp prin grafice interactive (tensiune arterială, puls, greutate, temperatură, saturație oxigen);
- **Programări online** cu selectarea medicului, datei și orei disponibile;
- **Chat medical AI** pentru obținerea de informații medicale generale, bazat pe modelul Llama 3.3 70B;
- **Mesagerie în timp real** cu medicii curanți și asistenții medicali;
- **Teleconsultații video** prin WebRTC, fără a necesita instalarea unor aplicații suplimentare;
- **Export GDPR complet** al tuturor datelor personale și medicale, în format HTML și JSON.

1.3.2 Funcționalități pentru medici

Medicii dispun de instrumente profesionale pentru:

- Gestionarea listei de pacienți atribuiți, cu acces rapid la fișa completă;
- Adăugarea și editarea înregistrărilor medicale (consultații, analize, tratamente, investigații, rețete);
- Vizualizarea și analiza semnelor vitale ale pacienților prin grafice de tendință;
- **Predicție risc AI** per pacient — un scor de risc de la 1 la 10, generat de modelul Llama 3.3 70B pe baza istoricului medical, cu recomandări clinice specifice;
- **Generare raport medical PDF** complet pentru fiecare pacient, bazat pe AI;
- Gestionarea programărilor zilei prin calendar integrat;
- Profil public accesibil pacienților, cu posibilitatea de a primi recenzii.

1.3.3 Funcționalități pentru asistenți medicali

Asistenții medicali pot:

- Gestiona programările (confirmare, anulare, reprogramare);
- Atribui pacienți la medici;
- Înregistra semne vitale pentru pacienți;
- Accesa statistici agregate despre pacienții clinicii.

1.3.4 Funcționalități administrative

Administratorii au acces la:

- Dashboard cu statistici globale ale platformei;
- Gestionarea completă a utilizatorilor (activare/dezactivare conturi, atribuire roluri);
- Vizualizarea tuturor programărilor din sistem;
- Jurnalul de audit al acțiunilor utilizatorilor.

1.4 Structura lucrării

Lucrarea este organizată în patru capitole principale:

Capitolul 1 — Introducere prezintă contextul și motivația alegerii temei, realizează un studiu comparativ cu soluțiile existente pe piață și descrie succint aplicația MediLink, contribuțiile originale și structura lucrării.

Capitolul 2 — Noțiuni teoretice și tehnologii utilizate prezintă fundamentele teoretice ale arhitecturilor web moderne și detaliază fiecare tehnologie utilizată în construirea MediLink: FastAPI, Angular 19, PostgreSQL, Redis, SQLAlchemy, JWT, WebSockets, WebRTC, Docker și Groq API.

Capitolul 3 — Contribuții personale constituie nucleul lucrării și prezintă în detaliu toate deciziile de proiectare și implementare: modelul de date, arhitectura REST API, sistemul de autentificare și securitate, implementarea fiecărui modul funcțional și soluțiile tehnice originale adoptate.

Capitolul 4 — Concluzii și perspective de dezvoltare sintetizează contribuțiile realizate, discută limitările actuale ale platformei și propune direcții de extindere și îmbunătățire pentru versiunile viitoare.

1.5 Contribuții originale

Principalele contribuții originale ale acestei lucrări sunt:

1. **Arhitectura integrată multi-rol** — proiectarea și implementarea unui sistem care servește simultan patru tipuri de utilizatori cu nevoi și permisiuni distincte, printr-un mecanism RBAC (*Role-Based Access Control*) granular implementat ca middleware FastAPI;

2. **Sistem de criptare transparentă a datelor medicale sensibile** — implementarea unui `TypeDecorator` SQLAlchemy personalizat (`EncryptedString`) care criptează automat câmpurile sensibile (CNP, diagnostic, tratament) la scriere și le decriptează la citire, fără modificări în codul de business;
3. **Integrarea AI pentru asistență medicală** — utilizarea modelului Llama 3.3 70B prin API-ul Groq pentru trei cazuri de utilizare distincte: chat medical contextual (cu memorie conversațională), predicție risc clinic per pacient și generare raport medical PDF;
4. **Teleconsultații video peer-to-peer** — implementarea unui sistem de apeluri video prin WebRTC cu semnalizare bazată pe WebSockets, fără necesitatea unui server de relay (TURN), utilizabil direct în browser;
5. **Sistem de notificări în timp real** — arhitectura bazată pe WebSockets pentru livrarea instantanee a notificărilor de sistem, confirmărilor de programare și alertelor de semne vitale anormale;
6. **Export GDPR automat** — generarea la cerere a unui document HTML complet și structurat cu toate datele personale și medicale ale unui utilizator, în conformitate cu articolul 20 din GDPR (*dreptul la portabilitatea datelor*);
7. **Grafice de tendință pentru semne vitale** — vizualizarea interactivă a evoluției temporale a parametrilor vitali cu detectarea automată a valorilor anormale și generarea de alerte.

2. Noțiuni teoretice și tehnologii utilizate

2.1 Arhitectura aplicațiilor web moderne

2.1.1 Modelul client-server și arhitectura REST

Aplicațiile web moderne sunt construite pe baza modelului **client-server**, în care responsabilitățile sunt clar separate: clientul (browserul web sau aplicația mobilă) se ocupă de prezentarea datelor și interacțiunea cu utilizatorul, în timp ce serverul gestionează logica de business, persistența datelor și securitatea.

Comunicarea dintre client și server se realizează prin interfețe de tip **API** (*Application Programming Interface*). Stilul arhitectural dominant în aplicațiile web actuale este **REST** (*Representational State Transfer*), definit de Roy Fielding în teza sa de doctorat din 2000 [10]. Principiile fundamentale ale REST includ:

- **Interfață uniformă** — resurse identificate prin URI-uri, manipulate prin metodele standard HTTP (GET, POST, PUT, DELETE, PATCH);
- **Statelessness** — fiecare cerere conține toate informațiile necesare procesării sale, serverul nu păstrează starea sesiunii între cereri;
- **Cacheability** — răspunsurile pot fi marcate ca cacheable sau non-cacheable;
- **Sistem pe niveluri** — clientul nu trebuie să știe dacă comunică direct cu serverul final sau cu un intermediar (load balancer, proxy).

MediLink implementează un API RESTful complet, cu peste 80 de endpoint-uri organizate logic pe resurse: /api/auth, /api/users, /api/patients, /api/doctors, /api/appointments, /api/medical-records, /api/vitals, /api/messages, /api/notifications și altele.

2.1.2 Arhitectura Single Page Application (SPA)

Frontendurile moderne sunt construite ca **Single Page Applications (SPA)**, în care navigarea între secțiuni nu implică reîncărcarea completă a paginii. Browserul descarcă inițial un set de resurse JavaScript, CSS și HTML, după care toate tranzițiile de navigare sunt gestionate de client, prin actualizarea dinamică a DOM-ului. Comunicarea cu serverul se face exclusiv prin apeluri API asincrone (fetch/XHR), fără reîncărcare de pagină.

Avantajele arhitecturii SPA includ o experiență de utilizare fluidă (similară aplicațiilor native), reducerea sarcinii pe server și posibilitatea de a construi interfețe complexe și reactive. Dezavantajele includ timpul inițial de încărcare mai mare și complexitatea adițională pentru SEO și indexarea motoarelor de căutare — aspecte irelevante pentru MediLink, care este o aplicație de uz intern, nu un site public.

2.2 FastAPI — Framework-ul backend

2.2.1 Prezentare generală

FastAPI [11] este un framework web modern pentru Python, creat de Sebastián Ramírez și lansat în 2018. Bazat pe standardul **ASGI** (*Asynchronous Server Gateway Interface*), FastAPI suportă nativ programarea asincronă prin cuvintele cheie `async/await` ale lui Python 3.7+, permițând gestionarea unui număr mare de conexiuni concurente cu un consum redus de resurse.

Conform benchmark-urilor independente [12], FastAPI se situează printre cele mai performante framework-uri web, cu viteze comparabile cu Go și Node.js, depășind semnificativ Django și Flask în scenariile de I/O-bound (acces la baze de date, apeluri externe).

2.2.2 Caracteristici distinctive

FastAPI se diferențiază prin câteva caracteristici esențiale pentru MediLink:

Validare automată prin Pydantic: Schema datelor se definește o singură dată, ca modele Python cu tipuri adnotate, iar FastAPI generează automat logica de validare, serializare și documentare. Orice cerere HTTP cu date invalide primește automat un răspuns 422 cu detalii despre câmpurile eronate.

```
1 from pydantic import BaseModel, EmailStr
2 from typing import Optional
3
4 class UserCreate(BaseModel):
5     email: EmailStr
6     password: str
7     first_name: str
8     last_name: str
9     role: str
10
11 class UserUpdate(BaseModel):
12     first_name: Optional[str] = None
13     last_name: Optional[str] = None
14     phone: Optional[str] = None
```

```
15 address: Optional[str] = None
```

Listing 2.1: Exemplu model Pydantic în MediLink

Documentație automată OpenAPI: FastAPI generează automat o interfață Swagger UI interactivă la `/api/docs` și o documentație ReDoc la `/api/redoc`, bazate pe specificația OpenAPI 3.0. Aceasta permite testarea API-ului direct din browser, fără instrumente suplimentare.

Dependency Injection: Sistemul de dependențe al FastAPI permite injectarea automată a resurselor comune (sesiunea bazei de date, utilizatorul autentificat, permisiunile RBAC) în funcțiile handler, eliminând codul boilerplate.

2.2.3 Dependency Injection în MediLink

MediLink utilizează extensiv sistemul de dependențe FastAPI pentru autentificare și control al accesului:

```
1 from fastapi import Depends, HTTPException
2 from app.middleware.auth import get_current_user
3 from app.core.database import get_db
4
5 @router.get("/medical-records/patient/{patient_id}")
6 async def get_patient_records(
7     patient_id: str,
8     current_user = Depends(get_current_user),
9     db: Session = Depends(get_db),
10 ):
11     if current_user.role not in ["doctor", "admin", "assistant"]:
12         raise HTTPException(403, "Acces interzis")
13     # ...
```

Listing 2.2: Dependency Injection pentru autentificare

2.3 Angular 19 — Framework-ul frontend

2.3.1 Prezentare generală

Angular [13] este un framework TypeScript complet pentru construirea de aplicații web, dezvoltat și menținut de Google. Spre deosebire de React sau Vue (care sunt biblioteci UI), Angular este un framework *opinionated*, care include soluții integrate pentru rutare, formulare, comunicare HTTP, testare și gestionarea stării.

MediLink folosește **Angular 19**, cea mai recentă versiune majoră, care introduce componente standalone (fără module NgModule), îmbunătățiri semnificative ale performanței prin Ivy

renderer și suport nativ pentru semnale reactive.

2.3.2 Componente Standalone

Arhitectura MediLink adoptă în întregime paradigma **componentelor standalone**, introdusă în Angular 14 și stabilizată în Angular 17. Fiecare componentă declară explicit dependențele sale (alte componente, directive, pipe-uri, module Angular Material), eliminând necesitatea NgModule-urilor și simplificând semnificativ structura aplicației:

```
1 @Component ({
2   selector: 'app-vitals',
3   standalone: true,
4   imports: [
5     CommonModule, MatCardModule, MatButtonModule,
6     NgApexchartsModule, TablerIconsModule,
7   ],
8   templateUrl: './vitals.html',
9 })
10 export class VitalsComponent implements OnInit {
11   vitals: VitalSign[] = [];
12
13   constructor(private api: ApiService, private auth: AuthService) {}
14
15   ngOnInit(): void {
16     this.loadVitals();
17   }
18 }
```

Listing 2.3: Componentă standalone Angular în MediLink

2.3.3 Angular Material Design 3

Interfața MediLink este construită pe **Angular Material** cu tema **Material Design 3** (M3), sistemul de design al Google. Componentele Material oferă accesibilitate built-in (ARIA attributes, navigare cu tastatura), suport pentru teme light/dark și un set consistent de variabile CSS (--mat-sys-primary, --mat-sys-surface, etc.) care se adaptează automat la tema selectată.

2.4 PostgreSQL — Baza de date

2.4.1 Prezentare generală

PostgreSQL [14] este un sistem de gestiune a bazelor de date relaționale (RDBMS) open-source, cu o istorie de dezvoltare de peste 35 de ani. Recunoscut pentru fiabilitate, conformitate strictă cu standardele SQL și un set bogat de funcționalități avansate, PostgreSQL este alegerea preferată pentru aplicații critice care necesită integritate a datelor.

MediLink utilizează PostgreSQL 16, beneficiind de:

- Tipul de date **UUID** nativ, utilizat ca cheie primară pentru toate entitățile;
- Tipul **JSONB** pentru stocarea datelor semi-structurate (lista de medicamente dintr-o prescripție, mesajele unui chat AI);
- **Tranzacții ACID** pentru garantarea integrității datelor în operațiile complexe;
- **Full-text search** pentru căutarea globală în platformă;
- Tipul **ENUM** pentru câmpuri cu valori predefinite (roluri utilizator, status programare).

2.4.2 SQLAlchemy ORM

SQLAlchemy [15] este cel mai utilizat ORM (*Object-Relational Mapper*) pentru Python, oferind o abstractizare completă a bazei de date prin maparea tabelor SQL la clase Python. MediLink utilizează SQLAlchemy 2.0 cu stilul declarativ:

```
1 class User(Base):
2     __tablename__ = "users"
3
4     id = Column(UUID(as_uuid=True), primary_key=True,
5                   default=uuid.uuid4)
6     email = Column(String, unique=True, nullable=False,
7                    index=True)
8     first_name = Column(String, nullable=True)
9     last_name = Column(String, nullable=True)
10    hashed_password = Column(String, nullable=True)
11    role = Column(Enum(UserRole), nullable=False)
12    phone = Column(String, nullable=True)
13    birth_date = Column(String, nullable=True)
14    mfa_secret = Column(String, nullable=True)
15    email_notifications = Column(Boolean, default=True)
16    is_active = Column(Boolean, default=True)
17    created_at = Column(DateTime(timezone=True),
```



```
17 server_default=func.now())
```

Listing 2.4: Model SQLAlchemy pentru entitatea User

2.4.3 Alembic — Migrații ale schemei

Gestionarea modificărilor schemei bazei de date se realizează prin **Alembic** [16], un instrument de migrații pentru SQLAlchemy. Fiecare modificare structurală a bazei de date (adăugare tabel, coloană nouă, index) este reprezentată ca un fișier de migrație Python versionat, cu funcții `upgrade()` și `downgrade()`, permițând evoluția controlată a schemei în producție.

2.5 Redis — Cache și sesiuni

Redis [17] (*Remote Dictionary Server*) este o bază de date în memorie de tip cheie-valoare, utilizată în MediLink pentru:

- Stocarea refresh token-urilor invalidate (blacklist pentru logout securizat);
- Cache pentru sesiunile WebSocket active;
- Gestionarea cozilor de task-uri pentru notificări asincrone.

Grație timpilor de acces sub milisecundă, Redis este soluția ideală pentru date volatile cu acces frecvent, fără a suprasolicita baza de date PostgreSQL.

2.6 Securitate

2.6.1 Autentificare JWT

JSON Web Token (JWT) [18] este un standard deschis (RFC 7519) pentru transmiterea securizată a informațiilor între părți ca obiect JSON semnat criptografic. MediLink implementează un flux de autentificare cu două token-uri:

- **Access token** — valabil 15 minute, transportat în header-ul HTTP `Authorization: Bearer <token>`, conține ID-ul utilizatorului și rolul acestuia;
- **Refresh token** — valabil 7 zile, stocat în cookie HTTP-Only (inaccesibil din JavaScript), utilizat exclusiv pentru obținerea unor noi access token-uri.

Algoritmul de semnare utilizat este **HS256** (HMAC-SHA256), cu o cheie secretă de minimum 256 de biți.

2.6.2 Autentificare în doi factori (2FA/TOTP)

MediLink implementează autentificarea în doi factori opțională bazată pe protocolul **TOTP** (*Time-based One-Time Password*), standardizat în RFC 6238 [19]. Implementarea este compatibilă cu aplicațiile Google Authenticator, Authy și Microsoft Authenticator.

Fluxul de activare 2FA implică: generarea unui secret TOTP pentru utilizator, afișarea unui cod QR scanabil, verificarea unui cod generat de aplicație pentru confirmare și stocarea secretului criptat în baza de date. La autentificare, dacă 2FA este activat, utilizatorul este redirecționat la pasul de verificare TOTP după introducerea credențialelor.

2.6.3 Criptarea datelor medicale sensibile

Datele medicale sensibile (diagnostice, tratamente, rezultate analize, CNP) sunt criptate în baza de date folosind **Fernet** — o implementare simetrică a AES-128-CBC cu HMAC-SHA256 pentru autentificare, din biblioteca *cryptography* pentru Python [20].

Criptarea este transparentă pentru codul de business, implementată printr-un **TypeDecorator SQLAlchemy** personalizat care interceptează operațiile de citire/scriere la nivel de coloană. Cheia de criptare este stocată exclusiv în variabile de mediu, niciodată în codul sursă sau baza de date.

2.6.4 RBAC — Control al accesului bazat pe roluri

Role-Based Access Control (RBAC) [21] este un model de control al accesului în care permisiunile sunt asociate rolurilor, iar utilizatorii primesc roluri. MediLink definește patru roluri: *admin*, *doctor*, *assistant* și *pacient*, fiecare cu un set specific de permisiuni pentru fiecare resursă a API-ului.

2.7 Inteligența artificială în medicină

2.7.1 Large Language Models (LLM)

Modelele lingvistice mari (LLM) sunt rețele neuronale de tip transformator [22] antrenate pe corpusuri masive de text, capabile să genereze text coerent, să răspundă la întrebări complexe și să realizeze raționamente în lanț. Modelul **Llama 3.3 70B** [23], dezvoltat de Meta AI, este utilizat în MediLink prin **API-ul Groq** [24], care oferă inferență LLM extrem de rapidă (150+ token-uri/secundă) prin hardware specializat LPU (*Language Processing Unit*).

2.7.2 Aplicații AI în MediLink

Chatbot medical contextual: Pacienții pot conversa cu un asistent AI medical care menține contextul conversației (ultimele 10 schimburi), are acces la profilul medical al pacientului (diagnostice cronice, alergii, medicație curentă) și este instruit printr-un prompt de sistem să ofere informații generale, să recomande consultul medical pentru situații urgente și să nu înlocuiască sfatul medicului.

Predicție risc clinic: Pentru fiecare pacient, medicul poate solicita o evaluare AI a riscului medical, pe o scală de la 1 la 10. Modelul primește ca context: vârsta și sexul pacientului, condițiile cronice, ultima medicamente prescrise, ultimele analize și semnele vitale recente. Răspunsul include scorul de risc, justificarea și recomandări clinice concrete.

Generator de raport PDF: La cererea medicului, AI-ul generează un raport medical narativ complet pentru un pacient, structurat pe secțiuni (sumar clinic, evoluție, plan terapeutic, recomandări), care este convertit în format PDF.

2.8 Comunicare în timp real

2.8.1 WebSockets

WebSockets [25] (RFC 6455) reprezintă un protocol de comunicare bidirecțional full-duplex peste o singură conexiune TCP persistentă. Spre deosebire de HTTP (care funcționează prin cerere-răspuns), WebSockets permit serverului să trimită date clientului în orice moment, fără ca acesta să facă o cerere prealabilă.

MediLink utilizează WebSockets pentru:

- Livrarea instantanee a notificărilor sistem (confirmare programare, alertă semne vitale, mesaj nou);
- Actualizarea în timp real a badge-ului cu notificări necitite din header;
- Semnalizarea pentru teleconsultațiile WebRTC (schimbul de SDP offers/answers și ICE candidates).

2.8.2 WebRTC — Teleconsultații video

WebRTC (*Web Real-Time Communication*) [26] este un set de standarde și API-uri web care permit comunicarea audio-video și schimbul de date direct între browsere (peer-to-peer), fără a necesita un server intermediar pentru fluxul media.

Procesul de stabilire a unei sesiuni WebRTC implică:

1. **Semnalizare** — schimbul de descriptori de sesiune SDP (*Session Description Protocol*) și candidați ICE (*Interactive Connectivity Establishment*) prin canalul WebSocket;
2. **Negocierea codecurilor** — browserele se pun de acord asupra codecurilor audio (Opus) și video (VP8/H.264) suportate;
3. **Traversarea NAT** — protocolul ICE găsește calea optimă de conectivitate (direct sau prin STUN);
4. **Transmiterea media** — fluxurile audio/video sunt criptate cu DTLS-SRTP și transmise peer-to-peer.

2.9 Containerizare cu Docker

Docker [27] este o platformă de containerizare care permite împachetarea aplicațiilor împreună cu toate dependențele lor într-o unitate izolată portabilă numită **container**. Spre deosebire de mașinile virtuale, containerele partajează kernel-ul sistemului de operare gazdă, oferind izolare cu un overhead minim.

Docker Compose permite definirea și orchestrarea unei stive multi-container printr-un singur fișier YAML (`docker-compose.yml`). Stack-ul MediLink este definit ca cinci servicii interconectate: `backend` (FastAPI), `frontend` (Angular + Nginx), `db` (PostgreSQL 16), `redis` (Redis 7) și `nginx` (reverse proxy).

Avantajele arhitecturii containerizate pentru MediLink includ: portabilitate completă (rulează identic pe orice sistem cu Docker), izolarea mediilor de dezvoltare și producție, scalabilitate orizontală și un proces de deployment simplificat la o singură comandă.

2.10 GDPR și protecția datelor medicale

Regulamentul General privind Protecția Datelor (GDPR) [28] (Regulamentul UE 2016/679) stabilește cadrul legal pentru procesarea datelor personale în Uniunea Europeană. Datele medicale sunt clasificate drept **date speciale** (articolul 9 GDPR), care beneficiază de protecție sporită și pot fi procesate doar în condiții stricte.

MediLink implementează cerințele GDPR prin:

- **Dreptul de acces** (Art. 15) — pacienții pot vedea toate datele stocate despre ei;
- **Dreptul la portabilitate** (Art. 20) — export complet în format HTML și JSON la cerere;
- **Dreptul la rectificare** (Art. 16) — utilizatorii pot actualiza profilul personal;

- **Minimizarea datelor** (Art. 5.1.c) — se stochează exclusiv datele necesare funcționării;
- **Pseudonimizare și criptare** (Art. 32) — datele sensibile sunt criptate în baza de date;
- **Consimțământ explicit** — pacienții confirmă consimțământul GDPR la înregistrare.

3. Contribuții personale

3.1 Arhitectura generală a sistemului

MediLink este construit pe o arhitectură modernă cu trei niveluri (*three-tier architecture*): prezentare (frontend Angular), logică de business (backend FastAPI) și persistență (PostgreSQL + Redis). Figura 3.1 ilustrează componentele principale și fluxul de comunicare dintre ele.

Figura 3.1: Arhitectura generală a platformei MediLink

Toate serviciile rulează ca containere Docker orchestrate prin Docker Compose, cu Nginx ca reverse proxy care rutează cererile HTTP/HTTPS și conexiunile WebSocket către serviciile corespunzătoare. Această arhitectură garantează că întreaga platformă poate fi pornită cu o singură comandă (`docker compose up --build`), indiferent de sistemul de operare gazdă.

Separarea clară a responsabilităților permite scalabilitate independentă: în cazul unei creșteri a numărului de utilizatori, se pot lansa instanțe multiple de backend (FastAPI suportă nativ concurența asincronă) fără modificări ale frontend-ului sau bazei de date.

3.2 Modelul de date

3.2.1 Entitățile principale

Schema bazei de date MediLink cuprinde 14 tabele principale, fiecare reprezentând o entitate distinctă din domeniul medical. Tabelul 3.1 prezintă entitățile și responsabilitatea fiecăreia.

Tabela 3.1: Entitățile bazei de date MediLink

Entitate	Responsabilitate
users	Toți utilizatorii platformei (admin, doctor, asistent, pacient)
doctors	Profilul extins al medicilor (specializare, CV, program)
patients	Profilul medical al pacienților (grup sanguin, alergii, boli cronice)
doctor_patient	Relația many-to-many medic–pacient
appointments	Programările medicale cu status și istoricul lor
medical_records	Fișele medicale (consultații, analize, tratamente, rețete)
vital_signs	Semnele vitale înregistrate în timp pentru fiecare pacient
prescriptions	Rețetele medicale cu lista de medicamente în format JSONB
messages	Mesajele schimbate între utilizatori
notifications	Notificările sistem pentru fiecare utilizator
reviews	Recenziile pacienților pentru medici
ai_conversations	Istoricul conversațiilor AI per pacient
audit_logs	Jurnalul de audit al acțiunilor importante
user_sessions	Sesiunile active (refresh tokens)

3.2.2 Cheile primare UUID

O decizie de proiectare importantă a fost utilizarea **UUID v4** ca cheie primară pentru toate entitățile, în locul ID-urilor întregi auto-incrementate convenționale. Avantajele acestei abordări în contextul medical sunt:

- **Securitate** — ID-urile UUID nu sunt predictibile, prevenind enumerarea resurselor (*IDOR attacks*);
- **Scalabilitate** — unicitatea globală permite generarea cheilor pe client sau în sisteme distribuite;
- **Integritate referențială** — mai ușor de verificat în auditare față de întregi.

3.2.3 Criptarea transparentă — EncryptedString

O contribuție tehnică originală este implementarea unui **TypeDecorator SQLAlchemy** personalizat pentru criptarea transparentă a câmpurilor sensibile:

```

1 from sqlalchemy.types import TypeDecorator, String
2 from cryptography.fernet import Fernet
3 import os
4

```

```
5 class EncryptedString(TypeDecorator):
6     impl = String
7     cache_ok = True
8
9     def __init__(self):
10         super().__init__()
11         key = os.environ.get("ENCRYPTION_KEY", "")
12         self._fernet = Fernet(key.encode()) if key else None
13
14     def process_bind_param(self, value, dialect):
15         """Criptare la scriere în DB"""
16         if value is None or self._fernet is None:
17             return value
18         return self._fernet.encrypt(value.encode()).decode()
19
20     def process_result_value(self, value, dialect):
21         """Decriptare la citire din DB"""
22         if value is None or self._fernet is None:
23             return value
24         try:
25             return self._fernet.decrypt(value.encode()).decode()
26         except Exception:
27             return value
```

Listing 3.1: TypeDecorator pentru criptare transparentă

Prin utilizarea `EncryptedString` ca tip de coloană SQLAlchemy, câmpurile sensibile (diagnostic, tratament, rezultate analize, CNP) sunt criptate automat la orice scriere în baza de date și decriptate transparent la orice citire, fără ca restul codului de business să fie conștient de acest mecanism.

3.3 API REST — Structura și implementarea

3.3.1 Organizarea endpoint-urilor

API-ul MediLink este organizat în 14 routere FastAPI, fiecare corespunzând unei resurse distincte. Prefixul `/api` este aplicat global la nivel de aplicație. Tabelul 3.2 prezintă principalele grupuri de endpoint-uri.

Tabela 3.2: Grupurile de endpoint-uri ale API-ului MediLink

Prefix	Metode	Descriere
/api/auth	POST	Login, refresh token, logout, activare 2FA
/api/me	GET, PUT	Profil utilizator autentificat
/api/patients	GET, POST, PUT, DELETE	Gestionare pacienți
/api/doctors	GET, POST, PUT	Gestionare medici și profiluri
/api/appointments	GET, POST, PUT, DELETE	Programări medicale
/api/medical-records	GET, POST, PUT, DELETE	Fișe medicale
/api/vitals	GET, POST	Semne vitale
/api/messages	GET, POST	Mesagerie
/api/notifications	GET, PUT	Notificări
/api/risk	GET	Predicție risc AI
/api/chat	POST, GET, DELETE	Chat medical AI
/api/search	GET	Căutare globală
/api/audit-log	GET	Jurnal de audit (admin)

3.3.2 Rate Limiting

Pentru protecția împotriva atacurilor de tip brute-force și DoS (*Denial of Service*), MediLink implementează **rate limiting** la două niveluri:

- **Nginx** — limită de 5 cereri/minut pentru endpoint-ul de login, 60 cereri/minut pentru API-ul general;
- **Aplicație** — biblioteca `slowapi` aplică limitări granulare per endpoint, cu identificare bazată pe IP.

3.4 Sistemul de autentificare și securitate

3.4.1 Fluxul de autentificare

Autentificarea în MediLink urmează un flux securizat în mai mulți pași, ilustrat în figura ??:

1. Utilizatorul trimite credențialele (email + parolă) prin POST la `/api/auth/login`;
2. Serverul verifică parola folosind **bcrypt** cu factor de cost 12;
3. Dacă 2FA este activat, serverul returnează un token temporar și cere codul TOTP;

4. La verificarea cu succes, serverul emite un access token JWT (15 min) și un refresh token HTTP-Only (7 zile);
5. La expirarea access token-ului, clientul apelează automat `/api/auth/refresh` pentru reînnoire, transparent pentru utilizator.

```

1 @router.post("/auth/login")
2 async def login(data: LoginRequest, db: Session = Depends(get_db)):
3     user = db.query(User).filter(User.email == data.email).first()
4     if not user or not verify_password(data.password,
5         user.hashed_password):
6         raise HTTPException(401, "ȚCredentiale invalide")
7
8     if not user.is_active:
9         raise HTTPException(403, "Cont dezactivat")
10
11     # ăDac 2FA este activat, ăreturneaz token temporar
12     if user.mfa_secret:
13         temp_token = create_temp_token(user.id)
14         return {"requires_mfa": True, "temp_token": temp_token}
15
16     # Altfel, emite token-urile complete
17     access_token = create_access_token(user.id, user.role)
18     refresh_token = create_refresh_token(user.id)
19     return {"access_token": access_token, "token_type": "bearer"}

```

Listing 3.2: Endpoint de login cu suport 2FA

3.4.2 Middleware-ul de autentificare

Verificarea token-ului JWT se realizează printr-un middleware FastAPI (`get_current_user`) injectat ca dependență în toate endpoint-urile protejate. Middleware-ul extrage token-ul din header-ul `Authorization`, îl verifică criptografic, verifică expirarea și returnează obiectul utilizator din baza de date.

3.4.3 Middleware RBAC

Controlul accesului granular este implementat prin funcții de dependență FastAPI specializate per rol:

```

1 def require_role(*roles: str):
2     def dependency(current_user = Depends(get_current_user)):
3         if current_user.role not in roles:
4             raise HTTPException(

```

```

5         status_code=403,
6         detail=f"Acces permis doar pentru: {'', ' '.join(roles)}"
7     )
8     return current_user
9     return dependency
10
11 # Utilizare în router
12 @router.post("/vitals/patient/{patient_id}")
13 async def add_vital(
14     patient_id: str,
15     data: VitalSignCreate,
16     current_user = Depends(require_role("assistant", "doctor")),
17     db: Session = Depends(get_db),
18 ):
19     ...

```

Listing 3.3: Middleware RBAC în MediLink

3.5 Modulul de fișe medicale

3.5.1 Tipologia înregistrărilor medicale

Fișa medicală electronică în MediLink este structurată în cinci tipuri de înregistrări, fiecare cu câmpuri specifice:

- **Consultație** (*consultatie*) — diagnostic, tratament, observații clinice;
- **Analiză de laborator** (*analiza*) — rezultatele analizelor, valori de referință, interpretare;
- **Tratament** (*tratament*) — schema terapeutică completă cu doze și durată;
- **Investigație paraclinică** (*investigatie*) — rezultate imagistice, EKG, spirometrie etc.;
- **Rețetă** (*reteta*) — lista medicamentelor prescrise cu doze și instrucțiuni.

3.5.2 Detectarea automată a anomaliilor

Fiecare înregistrare medicală poate fi marcată cu indicatorul `has_anomaly` și o notă descriptivă `anomaly_notes`. Această funcționalitate permite medicilor să evidențieze rapid înregistrările care necesită atenție (valori anormale de laborator, evoluție negativă) și pacienților să fie alertați prompt.

3.5.3 Paginarea și filtrarea

Lista de fișe medicale din interfața doctorului implementează **paginare manuală** pe o grilă de carduri, deoarece soluția standard MatPaginator din Angular Material funcționează exclusiv cu componenta mat-table. Soluția adoptată utilizează un MatTableDataSource pentru filtrare și sortare, combinat cu o proprietate calculată pagedRecords care aplică segmentarea *slice* pe datele filtrate:

```
1  pageSize = 10;
2  pageIndex = 0;
3
4  get pagedRecords(): MedicalRecord[] {
5      const filtered = this.dataSource.filteredData;
6      const start = this.pageIndex * this.pageSize;
7      return filtered.slice(start, start + this.pageSize);
8  }
9
10 get totalPages(): number {
11     return Math.ceil(this.dataSource.filteredData.length /
12         this.pageSize);
13 }
14 onFilterChange(value: string): void {
15     this.dataSource.filter = value.trim().toLowerCase();
16     this.pageIndex = 0; // reset la prima pagina la filtrare
17 }
```

Listing 3.4: Paginare manuală pentru grila de carduri

3.6 Modulul de semne vitale

3.6.1 Parametrii monitorizați

MediLink monitorizează șase tipuri de semne vitale, relevante clinic pentru evaluarea stării de sănătate:

- Tensiunea arterială sistolică și diastolică (mmHg);
- Pulsul cardiac (bătăi/minut);
- Temperatura corporală (°C);
- Saturația oxigenului din sânge — SpO2 (%);
- Greutatea corporală (kg).

3.6.2 Vizualizarea grafică

Graficele de evoluție temporală sunt implementate cu biblioteca **ApexCharts** [29] prin wrapper-ul Angular `ng-apexcharts`. Fiecare parametru vital este afișat pe un grafic de tip line-chart cu:

- Axă X temporală cu formatare automată a datelor;
- Zonă colorată sub curbă pentru vizualizare facilă a tendințelor;
- Tooltip interactiv cu valoarea exactă și data/ora măsurătorii;
- Suport complet pentru tema dark/light prin variabile CSS.

3.6.3 Alertele automate

Sistemul detectează automat valorile anormale prin compararea fiecărei măsurători cu intervalele de referință clinice. La detectarea unei anomalii, se generează automat o notificare de tip `warning` pentru pacient și medicul curant.

3.7 Sistemul de programări

3.7.1 Ciclul de viață al unei programări

O programare în MediLink trece prin mai multe stări definite ca *enum* PostgreSQL: `pending` (creată de pacient, în așteptarea confirmării), `confirmed` (confirmată de medic sau asistent), `completed` (consultația a avut loc) și `cancelled` (anulată de oricare parte).

Tranziția dintre stări este controlată strict pe server: pacienții pot anula programările proprii, dar nu le pot confirma; asistenții și medicii pot confirma sau anula; marcarea ca `completed` se face exclusiv de medic sau asistent.

3.7.2 Remindere automate

Un scheduler asincron (`APScheduler`) rulează la interval de 5 minute și verifică programările viitoare. Pacienții primesc:

- Un reminder notificare cu 24 de ore înainte;
- Un al doilea reminder cu 1 oră înainte.

Trimiterea email-ului de reminder este opțională și configurabilă de utilizator prin setarea `email_notifications`.

3.8 Inteligența artificială — Implementare detaliată

3.8.1 Chat medical AI

Chatbot-ul medical este implementat ca o sesiune de conversație cu memorie, bazată pe modelul Llama 3.3 70B:

```

1  async def generate_medical_response(
2      patient_context: str,
3      conversation_history: list,
4      user_message: str,
5  ) -> str:
6      system_prompt = f"""ȚEti un asistent medical AI pentru platforma
7          MediLink.
8      Ai acces la profilul medical al pacientului:
9      {patient_context}
10     Reguli:
11     - ȚOfer Ținformații medicale generale bazate pe dovezi
12     - Nu pune diagnostice definitive
13     - ȚRecomand consultarea medicului pentru probleme serioase
14     - ȚRspunde Țn limba ȚromȚn"""
15
16     messages = [{"role": "system", "content": system_prompt}]
17     # ȚAdaug ultimele 10 mesaje din istoric
18     for msg in conversation_history[-10:]:
19         messages.append({"role": msg["role"], "content":
20             msg["content"]})
21     messages.append({"role": "user", "content": user_message})
22
23     response = groq_client.chat.completions.create(
24         model="llama-3.3-70b-versatile",
25         messages=messages,
26         max_tokens=1024,
27         temperature=0.3,
28     )
29     return response.choices[0].message.content

```

Listing 3.5: Implementarea chat-ului medical AI

3.8.2 PredicȚia riscului medical

PredicȚia riscului este generată pe baza unui context clinic complet, construit din datele pacientului stocate Țn baza de date:

```

1  def build_risk_context(patient, records, vitals) -> str:

```

```

2     context = f"Vârsta: {calculate_age(patient)} ani\n"
3     context += f"Gen: {patient.gender}\n"
4     context += f"Boli cronice: {patient.chronic_conditions}\n"
5     context += f"Alergii: {patient.allergies}\n\n"
6
7     context += "Ultimele înregistrări medicale:\n"
8     for r in records[:5]:
9         context += f"- [{r.record_type}] {r.diagnosis or r.notes}\n"
10
11    context += "\nUltimele semne vitale:\n"
12    for v in vitals[:12]:
13        context += f"- {v.vital_type}: {v.value} {v.unit}\n"
14
15    return context

```

Listing 3.6: Construirea contextului pentru predicție risc

Răspunsul AI este structurat ca JSON cu câmpurile: `risk_score` (1-10), `risk_level` (scăzut/mediu/ridicat/critic), `factors` (lista factorilor de risc identificați) și `recommendations` (recomandări clinice specifice).

3.9 Teleconsultații video — WebRTC

3.9.1 Arhitectura semnalizării

Semnalizarea WebRTC în MediLink utilizează canalul WebSocket existent, cu un protocol de mesaje JSON definit prin tipuri de acțiuni: `call-offer`, `call-answer`, `ice-candidate`, `call-end`.

```

1  async initiateCall(targetUserId: string): Promise<void> {
2      this.peerConnection = new RTCPeerConnection({
3          iceServers: [{ urls: 'stun:stun.l.google.com:19302' }]
4      });
5
6      // Adaug fluxul local (ăcamer + microfon)
7      const stream = await navigator.mediaDevices.getUserMedia(
8          { video: true, audio: true }
9      );
10     stream.getTracks().forEach(track =>
11         this.peerConnection!.addTrack(track, stream)
12     );
13
14     // Trimite candidații ICE prin WebSocket
15     this.peerConnection.onicecandidate = (event) => {
16         if (event.candidate) {

```

```
17     this.ws.send(JSON.stringify({
18         type: 'ice-candidate',
19         to: targetUserId,
20         candidate: event.candidate
21     }));
22 }
23 };
24
25 // Creează și trimite oferta SDP
26 const offer = await this.peerConnection.createOffer();
27 await this.peerConnection.setLocalDescription(offer);
28 this.ws.send(JSON.stringify({
29     type: 'call-offer', to: targetUserId, sdp: offer
30 }));
31 }
```

Listing 3.7: Inițierea unui apel WebRTC în Angular

3.10 Mesageria în timp real

3.10.1 Arhitectura WebSocket

Conexiunea WebSocket este menținută pe toată durata sesiunii utilizatorului, reutilizând același canal pentru notificări, mesagerie și semnalizare WebRTC. La conectare, serverul înregistrează conexiunea asociată ID-ului utilizatorului într-un dicționar în memorie.

3.10.2 Serviciul Angular de WebSocket

Pe frontend, conexiunea WebSocket este gestionată de un serviciu Angular singleton (`WebSocketService`), care:

- Menține o singură conexiune pe sesiune, reconectând automat la deconectare;
- Expune un `Subject` RxJS observable pentru distribuirea mesajelor primite;
- Gestionează autentificarea conexiunii prin trimiterea token-ului JWT la conectare.

3.11 Exportul GDPR

La cererea pacientului, platforma generează un document HTML complet cu toate datele personale și medicale stocate. Documentul este structurat pe secțiuni: date personale, profil

medical (alergii, boli cronice, grup sanguin), istoricul programărilor, fișele medicale complete (cu datele decriptate), semnele vitale, prescripțiile și mesajele. Exportul este disponibil și în format JSON structurat, pentru interoperabilitate cu alte sisteme.

3.12 Interfața cu utilizatorul

3.12.1 Design system — Angular Material M3

Interfața MediLink este construită pe componentele Angular Material cu tema Material Design 3. Un aspect important al implementării este suportul complet pentru **modul întunecat** (dark mode), obținut prin utilizarea exclusivă a variabilelor CSS semantice (`--mat-sys-primary`, `--mat-sys-surface`, `--mat-sys-on-surface`) în locul culorilor hexadecimale hardcodate. Comutarea între moduri se face instantaneu, fără reîncărcarea paginii.

3.12.2 Diagrama fluxului de navigare

Navigarea în MediLink este controlată de un sistem de garduri de rută (*route guards*) care verifică autentificarea și rolul utilizatorului:

- Utilizatorii neautentificați sunt redirecționați automat la pagina de login;
- Fiecare rol accesează exclusiv dashboard-ul și funcționalitățile corespunzătoare;
- Accesul direct la URL-uri restricționate returnează pagina 403.

3.12.3 Responsive design

Toate paginile MediLink sunt complet responsive, adaptate pentru ecrane de la 320px (telefon mobil) până la 1920px (desktop full HD). Sistemul de grid utilizat este CSS Grid și Flexbox, cu breakpoints definite în SCSS.

3.13 Testarea aplicației

3.13.1 Teste unitare backend

Suita de teste backend este construită cu **pytest** [30] și include 60+ teste unitare și de integrare care acoperă:

- Autentificarea și gestionarea token-urilor;

- CRUD pentru toate resursele principale;
- Validarea schema-elor Pydantic;
- Controlul accesului RBAC;
- Criptarea/decriptarea datelor sensibile.

Baza de date utilizată în teste este o instanță SQLite în memorie, izolată de producție, cu rate limiting dezactivat prin variabila de mediu TESTING=1.

3.13.2 Rularea testelor

```
1 # pytest.ini
2 [pytest]
3 testpaths = tests
4 asyncio_mode = auto
5 env =
6     TESTING=1
7     DATABASE_URL=sqlite:///./test.db
8     SECRET_KEY=test-secret-key
9     ENCRYPTION_KEY=
```

Listing 3.8: Configurarea pytest pentru MediLink

4. Concluzii și perspective de dezvoltare

4.1 Concluzii

Lucrarea de față a prezentat proiectarea și implementarea platformei MediLink — o soluție software completă pentru digitalizarea activității medicale, cu accent pe securitate, conformitate GDPR și integrarea inteligenței artificiale.

Prin parcurgerea etapelor de analiză, proiectare și implementare, au fost atinse toate obiectivele propuse inițial:

- S-a implementat un **ecosistem medical integrat** care acoperă fluxul complet al activității clinice: de la programare și consultație, la prescripție, monitorizare semne vitale și teleconsultație;
- **Securitatea datelor medicale sensibile** a fost tratată ca cerință de prim rang, nu ca o adăugire ulterioară — criptarea Fernet, autentificarea JWT cu 2FA și mecanismul RBAC granular asigurând o protecție în profunzime;
- **Integrarea AI** a adus valoare practică concretă: chatbot-ul medical oferă pacienților acces instant la informații medicale generale, predicția de risc îi ajută pe medici să prioritizeze cazurile, iar raportul PDF automat reduce semnificativ timpul administrativ;
- **Comunicarea în timp real** prin WebSockets și teleconsultațiile WebRTC au transformat platforma dintr-un simplu sistem de gestionare a datelor într-un ecosistem de comunicare medicală activă;
- **Arhitectura Docker** a garantat portabilitatea și reproductibilitatea completă a soluției — întreaga platformă poate fi pornită pe orice mașină cu o singură comandă.

Comparativ cu soluțiile existente analizate în Capitolul 1, MediLink se distinge prin combinarea unică a tuturor acestor funcționalități într-o singură platformă open-source, containerizată, accesibilă și conformă cu standardele europene de protecție a datelor.

Din punct de vedere tehnic, proiectul a demonstrat eficiența combinației **FastAPI + Angular 19 + PostgreSQL**: FastAPI pentru viteza de dezvoltare și performanța I/O-asincronă, Angular pentru construirea de interfețe complexe și reactive, iar PostgreSQL pentru fiabilitatea datelor critice. Utilizarea componentelor standalone Angular și a dependency injection FastAPI a condus la un cod modular, testabil și ușor de extins.

4.2 Limitări actuale

În stadiul actual, MediLink prezintă câteva limitări care reprezintă oportunități de îmbunătățire pentru versiunile viitoare:

- **Infrastructura de semnalizare WebRTC** folosește un server WebSocket centralizat pentru schimbul de mesaje de semnalizare; în scenarii cu mii de apeluri simultane, această componentă ar putea deveni un bottleneck și ar necesita scalare orizontală;
- **Stocarea media** — platforma nu include în prezent funcționalități de stocare a fișierelor atașate fișelor medicale (imagini DICOM, PDF-uri de investigații); aceasta reprezintă o limitare semnificativă față de sistemele EHR enterprise;
- **Integrarea cu sistemele de asigurări** — MediLink nu este integrat cu sistemele CNAS sau ale asigurătorilor privați, ceea ce limitează adoptarea în cabinetele care lucrează cu decontare CNAS;
- **Aplicație mobilă nativă** — platforma este accesibilă pe mobil prin browser (responsive design), dar o aplicație nativă iOS/Android ar oferi o experiență mai bună, notificări push native și acces la senzori (pulsoximetru Bluetooth etc.);
- **Suport multi-tenant** — arhitectura actuală este concepută pentru o singură unitate medicală; scalarea la nivel de rețea de clinici ar necesita implementarea izolării datelor la nivel de organizație.

4.3 Perspective de dezvoltare

Pe baza arhitecturii actuale, MediLink poate fi extins în mai multe direcții valoroase:

4.3.1 Integrarea dispozitivelor medicale IoT

Wearable-urile medicale moderne (smartwatch-uri cu EKG, glucometre Bluetooth, pulsoxime-trele IoT) pot trimite date direct în platforma MediLink prin API-ul de semne vitale. Integrarea cu standardul **HL7 FHIR** (*Fast Healthcare Interoperability Resources*) [31] ar permite interoperabilitatea cu orice dispozitiv medical certificat.

4.3.2 Diagnosticare asistată AI prin imagini medicale

Integrarea unui model de viziune computațională (de exemplu, prin API-ul OpenAI Vision) ar permite analiza automată a imaginilor medicale atașate fișelor: interpretarea radiografiilor,

detecția leziunilor dermatologice din fotografii, analiza EKG-urilor digitizate.

4.3.3 Analiză predictivă populațională

Datele agregate (anonimizate) ale platformei pot fi folosite pentru modele de analiză predictivă la nivel de populație: identificarea tendințelor epidemiologice locale, predicția sezonaliității bolilor, optimizarea planificării resurselor medicale.

4.3.4 Aplicație mobilă

Dezvoltarea unei aplicații mobile native cu **Flutter** [32] (care permite generarea de aplicații iOS și Android dintr-o singură bază de cod) ar extinde semnificativ accesibilitatea platformei, adăugând notificări push, acces la camera pentru teleconsultații optimizate și integrare cu senzori Bluetooth.

4.3.5 Conformitate HL7 FHIR și Spațiul European al Datelor de Sănătate

Implementarea standardului HL7 FHIR pentru exportul și importul datelor medicale ar permite MediLink să devină interoperabil cu alte sisteme EHR și să se integreze în inițiativa **European Health Data Space** (EHDS) [33], care va deveni obligatorie în statele membre UE până în 2027.

4.3.6 Sistem de telemedicină asincronă

Extinderea platformei cu funcționalități de telemedicină asincronă — trimiterea de fotografii, filmulețe sau înregistrări audio pentru evaluare medicală ulterioară, fără a fi necesară o sesiune sincronă — ar crește semnificativ accesibilitatea pentru pacienții din zone rurale.

În concluzie, MediLink demonstrează că o platformă medicală modernă, completă și securizată poate fi construită cu tehnologii open-source, la un nivel de complexitate și calitate comparabil cu soluțiile comerciale, constituind o bază solidă pentru o continuare academică (proiect de masterat, doctorat) sau o aplicație cu potențial real de adoptare în sistemul de sănătate românesc.

Bibliografie

- [1] World Health Organization, *Global Strategy on Digital Health 2020–2025*, <https://www.who.int/docs/default-source/documents/g4dhdaa2a9f352b0445bafbc79ca799dce4.pdf>, Accesat: mai 2026, 2023.
- [2] European Commission, *European Health Data Space — Proposal for a Regulation*, https://health.ec.europa.eu/ehealth-digital-health-and-care/european-health-data-space_en, Accesat: mai 2026, 2021.
- [3] Ministerul Sănătății, *Strategia națională de sănătate 2023–2030*, <https://www.ms.ro/ro/strategie-nationala-de-sanatate/>, Accesat: mai 2026, 2023.
- [4] Karan Singhal, Shekoofeh Azizi, Tao Tu et al., „Large language models encode clinical knowledge”, în *Nature* 620 (2023), pp. 172–180, DOI: [10.1038/s41586-023-06291-2](https://doi.org/10.1038/s41586-023-06291-2).
- [5] Epic Systems, *Epic — Electronic Health Records*, <https://www.epic.com>, Accesat: mai 2026, 2024.
- [6] Doctolib, *Doctolib — Plateforme de santé numérique*, <https://www.doctolib.fr>, Accesat: mai 2026, 2024.
- [7] Practo Technologies, *Practo — Healthcare Platform*, <https://www.practo.com>, Accesat: mai 2026, 2024.
- [8] Hipocrate Software, *Hipocrate — Software medical pentru cabinete*, <https://www.hipocrate.ro>, Accesat: mai 2026, 2024.
- [9] MedicalSoft, *MedicalSoft — Soluții software pentru sănătate*, <https://www.medicalsoft.ro>, Accesat: mai 2026, 2024.
- [10] Roy Thomas Fielding, „Architectural Styles and the Design of Network-based Software Architectures”, Teză de doct., University of California, Irvine, 2000.
- [11] Sebastián Ramírez, *FastAPI — Modern, fast web framework for building APIs with Python*, <https://fastapi.tiangolo.com>, Accesat: mai 2026, 2024.
- [12] TechEmpower, *Web Framework Benchmarks — Round 22*, <https://www.techempower.com/benchmarks/>, Accesat: mai 2026, 2024.
- [13] Google LLC, *Angular — The web development framework for building the future*, <https://angular.dev>, Accesat: mai 2026, 2024.
- [14] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, <https://www.postgresql.org/docs/16/>, Accesat: mai 2026, 2024.

- [15] Mike Bayer et al., *SQLAlchemy — The Python SQL Toolkit and Object Relational Mapper*, <https://www.sqlalchemy.org>, Accesat: mai 2026, 2024.
- [16] Mike Bayer, *Alembic — Database migration tool for SQLAlchemy*, <https://alembic.sqlalchemy.org>, Accesat: mai 2026, 2024.
- [17] Redis Ltd., *Redis — In-memory data structure store*, <https://redis.io>, Accesat: mai 2026, 2024.
- [18] Michael Jones, John Bradley și Nat Sakimura, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015, doi: [10.17487/RFC7519](https://doi.org/10.17487/RFC7519).
- [19] David M'Raihi, Salah Machani, Mingliang Pei și Johan Rydell, *TOTP: Time-Based One-Time Password Algorithm*, RFC 6238, IETF, 2011, doi: [10.17487/RFC6238](https://doi.org/10.17487/RFC6238).
- [20] pyca/cryptography contributors, *cryptography — Cryptographic recipes and primitives for Python*, <https://cryptography.io/en/latest/>, Accesat: mai 2026, 2024.
- [21] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein și Charles E. Youman, „Role-Based Access Control Models”, în *IEEE Computer*, vol. 29, 2, 1996, pp. 38–47.
- [22] Ashish Vaswani, Noam Shazeer, Niki Parmar et al., „Attention Is All You Need”, în *Advances in Neural Information Processing Systems* 30 (2017).
- [23] Meta AI, *Llama 3: Open Foundation and Fine-Tuned Chat Models*, <https://ai.meta.com/llama/>, Accesat: mai 2026, 2024.
- [24] Groq, Inc., *Groq API — Fast AI Inference*, <https://console.groq.com>, Accesat: mai 2026, 2024.
- [25] Ian Fette și Alexey Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011, doi: [10.17487/RFC6455](https://doi.org/10.17487/RFC6455).
- [26] W3C and IETF, *WebRTC 1.0: Real-Time Communication Between Browsers*, <https://www.w3.org/TR/webrtc/>, Accesat: mai 2026, 2024.
- [27] Docker, Inc., *Docker — Accelerated Container Application Development*, <https://www.docker.com>, Accesat: mai 2026, 2024.
- [28] Parlamentul European și Consiliul Uniunii Europene, *Regulamentul (UE) 2016/679 — Regulamentul General privind Protecția Datelor*, <https://eur-lex.europa.eu/legal-content/RO/TXT/?uri=CELEX:32016R0679>, Publicat în Jurnalul Oficial al UE L 119, 4 mai 2016, 2016.
- [29] ApexCharts, *ApexCharts — Modern & Interactive Open-source Charts*, <https://apexcharts.com>, Accesat: mai 2026, 2024.
- [30] pytest contributors, *pytest — Full-featured Python testing tool*, <https://docs.pytest.org>, Accesat: mai 2026, 2024.

- [31] HL7 International, *HL7 FHIR R4 — Fast Healthcare Interoperability Resources*, <https://www.hl7.org/fhir/>, Accesat: mai 2026, 2024.
- [32] Google LLC, *Flutter — Build apps for any screen*, <https://flutter.dev>, Accesat: mai 2026, 2024.
- [33] European Commission, *European Health Data Space — Regulation Proposal COM/2022/197*, https://health.ec.europa.eu/system/files/2022-05/com_2022_197_en.pdf, Accesat: mai 2026, 2022.